Calgary Property Value Prediction

Project Report - Regression, KNN, Random Forest and Gradient Boosting Models

Chris Petrauskas (UCID: 30270696)
Javier Becaria (UCID: 30234458)

Abstract

This project uses parcel-level property assessment data from the City of Calgary to model value per square metre. After cleaning and sampling a large multi-year dataset, features were engineered to capture both property characteristics and local market context. Regularized linear models were first used to establish a baseline and identify key drivers, followed by a KNN regressor that treats valuation as a local market problem by leveraging nearby comparable properties. Results are consistent across methods: location and immediate market context dominate. The best KNN model (k = 5, distance-weighted) achieved a validation R^2 of 0.832 versus ~0.720 for ridge and lasso, suggesting property values are best understood as part of a local spatial system rather than in isolation.

1. Problem Definition and Motivation

The goal is to predict value per square metre and understand its key drivers. Using value per square metre normalizes for property size, making comparisons across parcels more meaningful. The central question is whether value is driven primarily by structural features (e.g., age, type) or by location and surrounding market conditions. This problem is well-suited to both urban economics and machine learning, as it enables a direct comparison between global models (regularized regression) and local models (KNN).

2. Data Sources and Preparation

2.1 Dataset

The dataset comes from the City of Calgary Open Data Portal and contains parcel-level historical property assessment records. The full raw source is very large, so the workflow uses a cleaned and sampled version suitable for modeling. The working dataset used in the notebook contains 117,612 observations after preparation, split into 94,089 training rows, 11,761 validation rows, and

11,762 test rows. We first undertook cross sectional analysis on 2025 valuations only before extending our analysis to the last 5 years.

Item	Details
Source	City of Calgary Historical Property Assessments
Unit of analysis	A property parcel observation
Target variable	VALUE_PER_SM (assessed value per square metre)
Working sample size	117,612 observations
Train / validation / test	94,089 / 11,761 / 11,762

2.2 Cleaning and Feature Engineering

The notebook cleans the data by removing invalid or extreme values, standardizing types, and keeping only records suitable for modeling. Several engineered features were then created to represent both parcel attributes and local market structure.

- VALUE_PER_SM as the main target variable.
- AGE_OF_PROPERTY derived from construction year.
- center_latitude and center_longitude extracted from parcel geometry to capture location.
- DIST_FROM_CENTRE_KM to capture broad centrality.
- COMM_PROPERTY_COUNT as a density proxy for the surrounding community.
- NEARBY_MEDIAN_VALUE_PER_SM as a local market benchmark based on nearby properties.

3. Exploratory Data Analysis

Before fitting models, the visualizations already suggest that spatial structure matters a great deal. Values vary meaningfully across the city, and local density appears related to typical value levels.

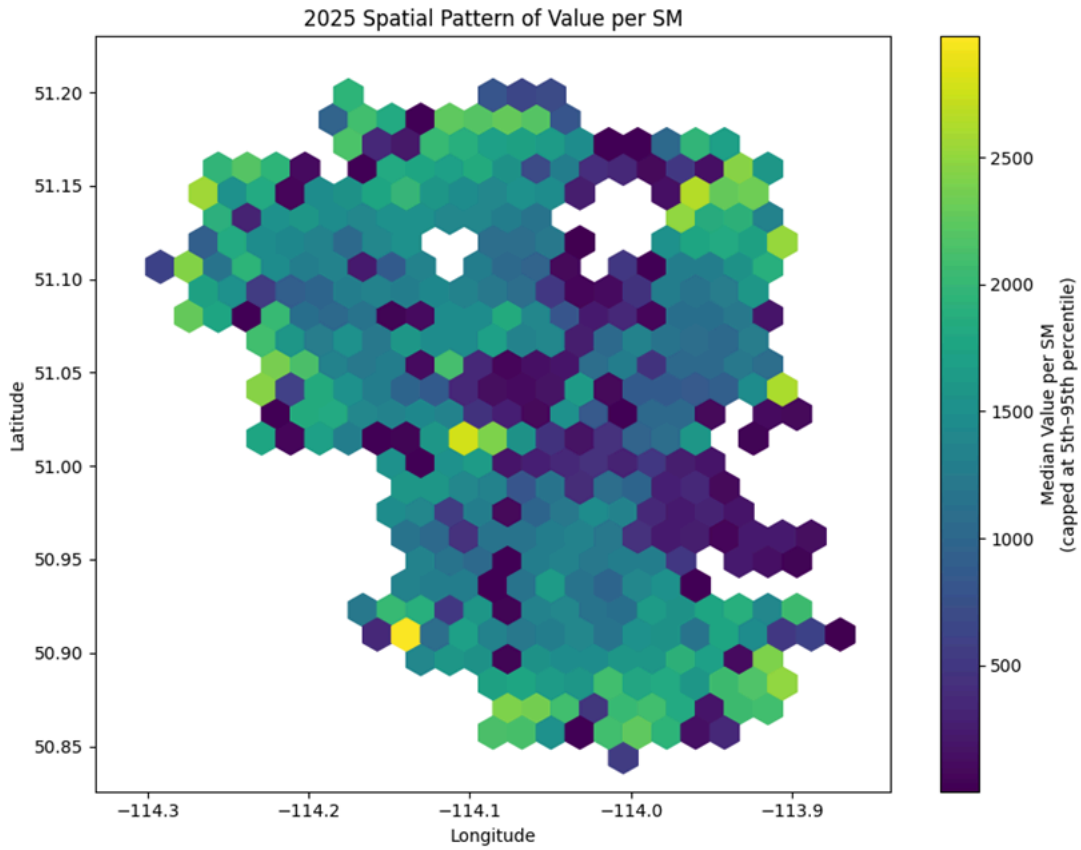

Figure 1. 2025 spatial pattern of value per square metre (hexbin map).

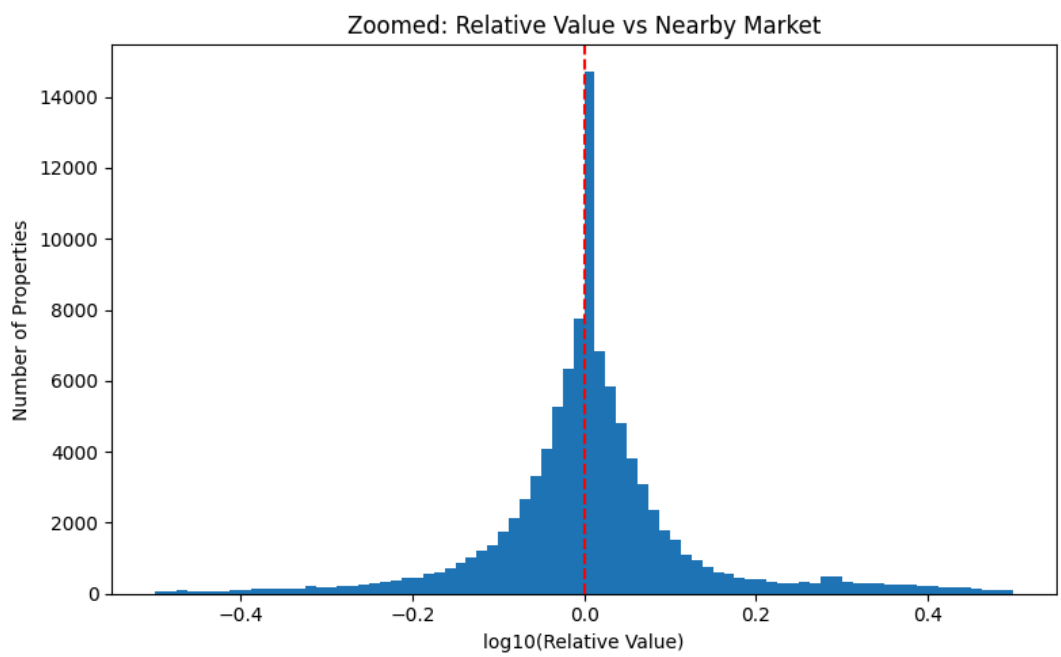

Figure 1.1 Property value per square metre against median value of 10 nearest properties.

Interpretation: The map and graph show clear geographic clustering rather than a random spread of values. Higher-value and lower-value areas form spatial pockets, which is an early sign that location is likely to be one of the dominant drivers in the models.

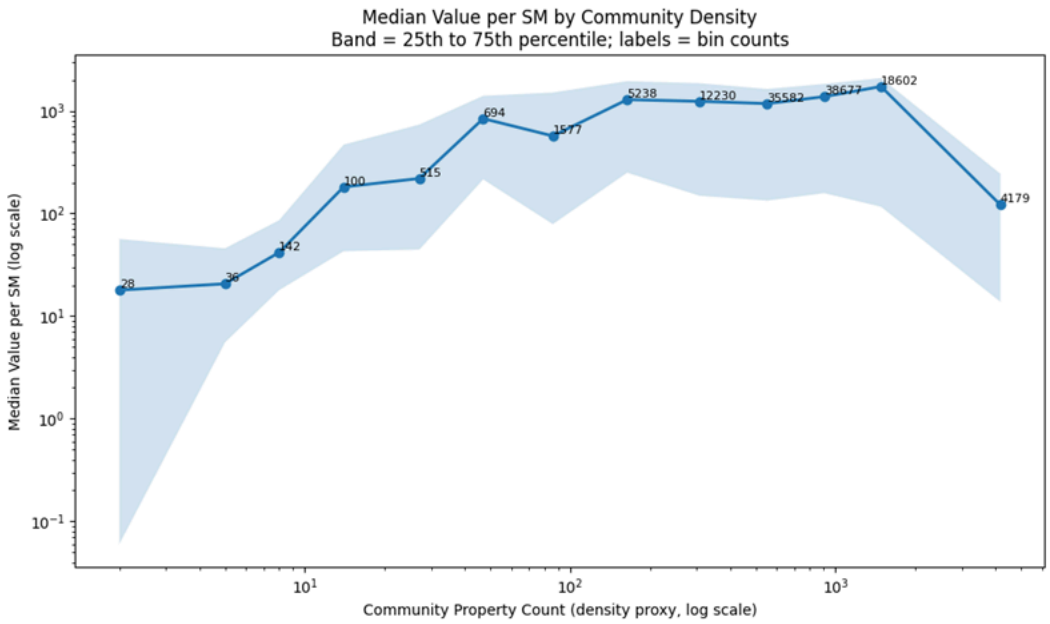

Figure 2. Median value per square metre by community density.

Interpretation: Median value tends to rise as the density proxy increases, at least through much of the range. This suggests that local market context is not just about exact coordinates, but also about the type of surrounding urban environment.

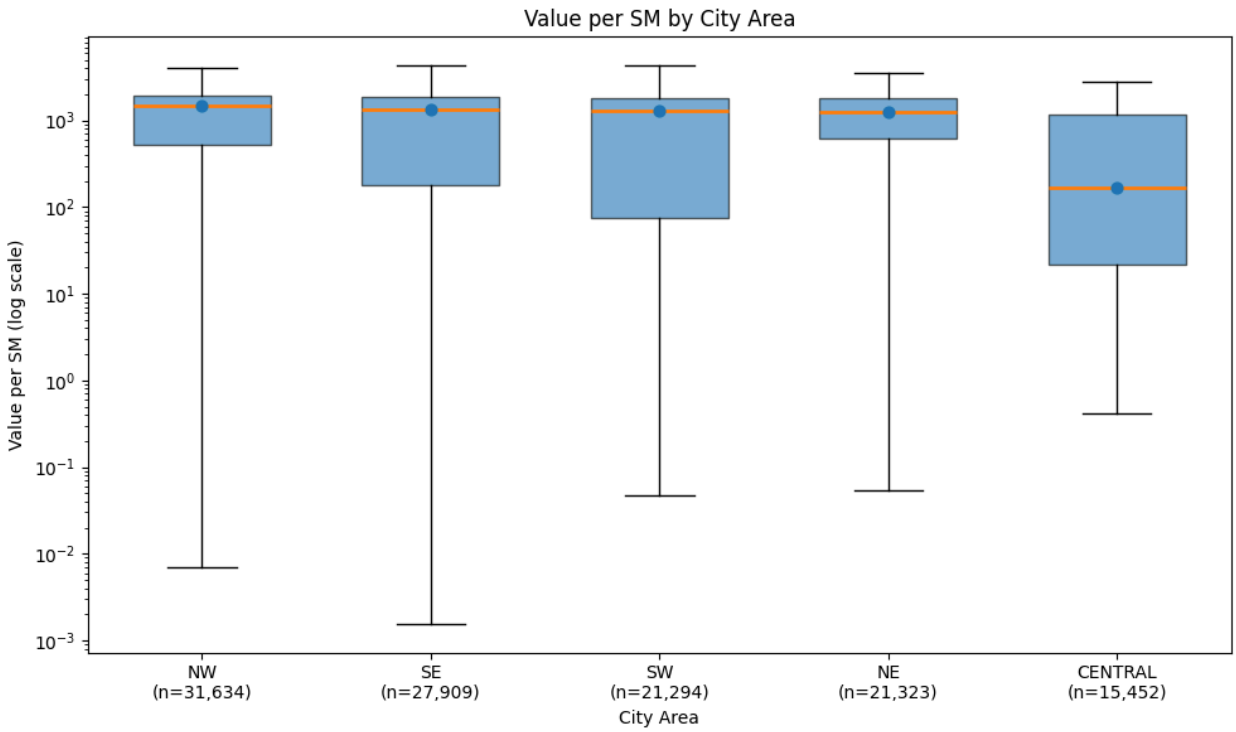

Figure 2.1 Value by quadrant.

Interpretation: We see that surprisingly the central “quadrant” has lower value per square metre. Likely because this one includes areas surrounding the downtown core as well as the core itself. Surprisingly the other quadrants see little variation although this could change based on the methodology used for each sector.

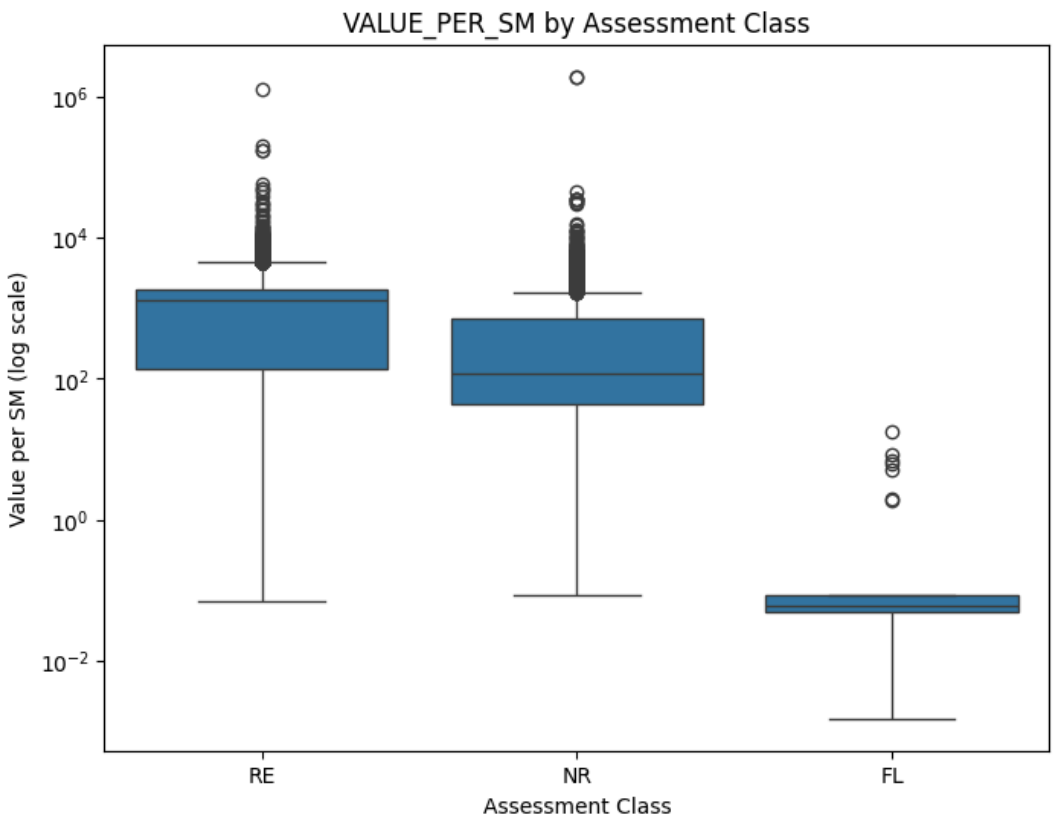

Figure 2.2 Median value per square metre by property type.

Interpretation: Property value tends to be highest in residential contexts. Non residential (commercial space) is slightly lower and farmland (which only makes up a small portion of the properties assessed) has the lowest value typically.

4. Regression and KNN Models

4.1 Baseline regression models

Ridge and lasso regression were used as baseline models. These models are useful because they are easy to interpret, handle mixed numeric and categorical data cleanly after preprocessing, and provide a clear benchmark before moving to a more local, nonparametric method.

For the regression baseline, the model used AGE_OF_PROPERTY, DIST_FROM_CENTRE_KM, LOG_COMM_PROPERTY_COUNT, LOG_NEARBY_MEDIAN_VALUE_PER_SM, ASSESSMENT_CLASS, PROPERTY_TYPE, and CITY_AREA. Numeric variables were

median-imputed and standardized, while categorical variables were imputed with the most frequent category and encoded for modeling.

The response variable was specified as log(VALUE_PER_SM) to improve the suitability of the data for linear modeling. Property values are typically right-skewed with greater variability at higher price levels, and taking the log compresses this scale, stabilizing variance and reducing the influence of extreme high-value observations. In addition, many underlying relationships are multiplicative rather than additive, and the log transformation makes these relationships more linear, aligning better with the assumptions of Ridge and Lasso regression and leading to more stable, interpretable, and generalizable results.

4.2 KNN model

The KNN model was chosen because property valuation often works by reference to comparable nearby properties. Rather than imposing one global functional form, KNN predicts from the most similar observations in the training set.

The KNN specification used AGE_OF_PROPERTY, center_latitude, center_longitude, ASSESSMENT_CLASS, and PROPERTY_TYPE. Numeric variables were median-imputed and standardized; categorical variables were imputed and one-hot encoded. The target was again log(VALUE_PER_SM). The model was estimated with distance weighting, and k was tuned over the set {3, 5, 10, 15, 25, 35, 50} using validation performance.

4.3.1 Baseline regression performance

The ridge and lasso models produced nearly identical results. That is useful in itself: it suggests the baseline story is stable and not very sensitive to the regularization choice.

Model	Best alpha	Train R-squared	Validation R-squared	Validation RMSE (log)
Ridge (L2)	0.078	0.742	0.719	1.193
Lasso (L1)	0.0001	0.742	0.720	1.193

These models perform reasonably well and confirm that the engineered features contain real predictive information. However, their fit also leaves room for improvement, which is what we would expect if valuation is driven by highly local relationships that a global linear structure cannot capture perfectly.

4.3.2 KNN tuning and model performance

KNN improved materially on the regression baseline. The best model used k = 5 with distance weighting, which means the prediction leaned most heavily on the closest comparable observations.

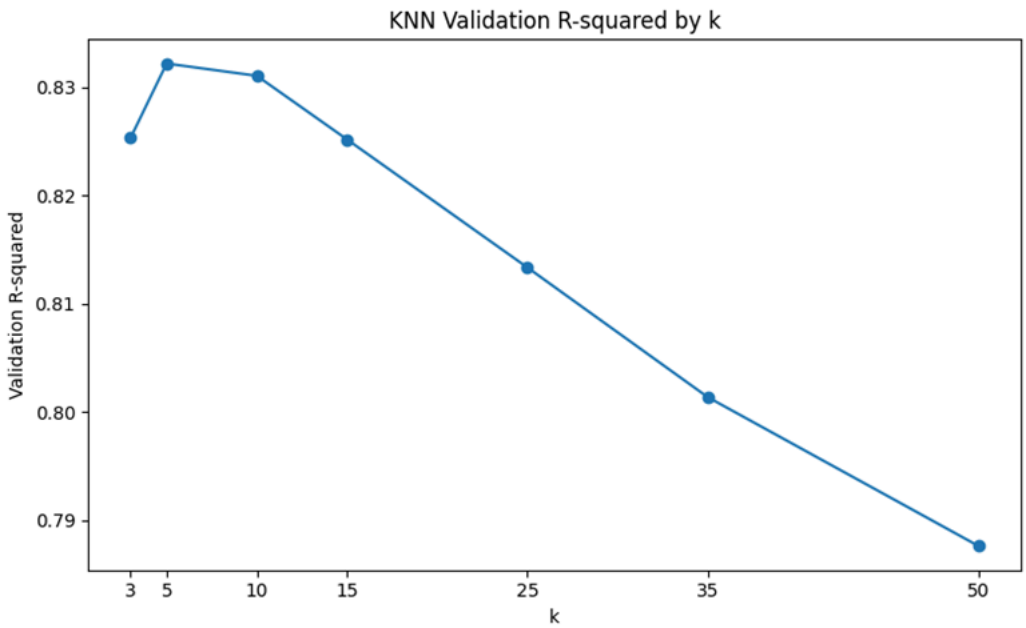

Figure 3. KNN validation R-squared by k.

Interpretation: Validation performance peaks at k = 5 and then gradually declines as the neighbourhood becomes too large. That pattern fits the intuition of a local market: averaging over too many neighbours starts to wash out the most relevant nearby information.

k	Train R-squared	Validation R-squared	Train RMSE (log)	Validation RMSE (log)	Validation RMSE (orig.)
3	0.874	0.825	0.807	0.941	872.144
5	0.887	0.832	0.764	0.923	673.199
10	0.897	0.831	0.731	0.926	699.910
15	0.900	0.825	0.720	0.942	728.163
25	0.901	0.813	0.713	0.973	781.285
35	0.902	0.801	0.711	1.003	-
50	0.902	0.788	0.710	1.038	-

The best KNN model achieved a validation R-squared of 0.832, compared with roughly 0.720 for the regression models. That is a meaningful improvement. It indicates that much of the remaining explainable variation comes from local nonlinear structure rather than from a more complex global coefficient pattern.

4.3.3 Permutation importance for the best KNN model

Feature	Importance	Std. dev.
center_latitude	1.185	0.009
center_longitude	1.183	0.012
AGE_OF_PROPERTY	1.107	0.013
ASSESSMENT_CLASS	0.079	0.004
PROPERTY_TYPE	0.060	0.002

The permutation importance results reinforce the same story. Latitude and longitude are by far the most influential inputs, with age a distant next feature and the categorical descriptors contributing much less. In other words, once the model knows where a parcel is, much of the valuation problem is already being explained.

4.3.4 Model Evaluation

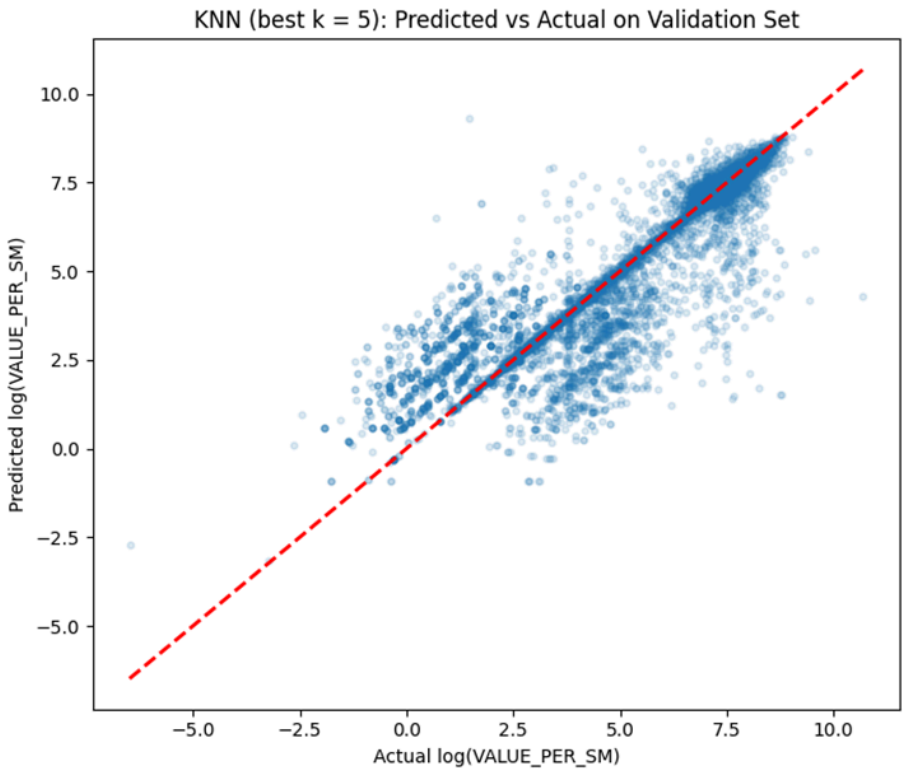

Figure 4. Predicted versus actual log(VALUE_PER_SM) for the best KNN model on the validation set.

Interpretation: The points cluster closely around the 45-degree line, which indicates that the best KNN model tracks observed values fairly well across much of the range. There is still visible dispersion, especially away from the densest band, but the overall fit is clearly strong.

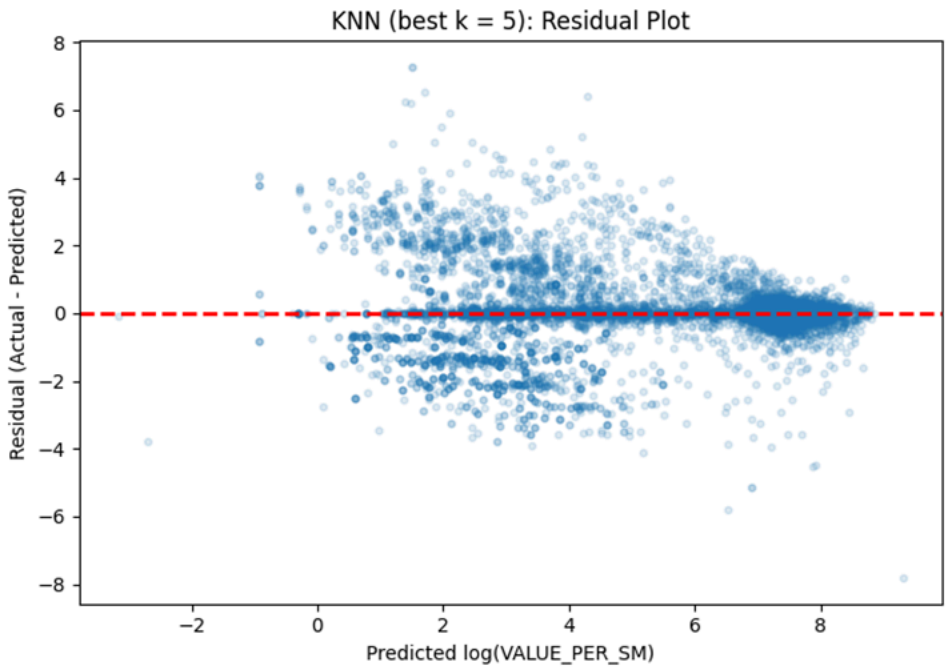

Figure 5. Residual plot for the best KNN model on the validation set.

Interpretation: Residuals are centred around zero but show uneven spread, indicating generally good performance with weaker accuracy in noisier parts of the distribution.

Overall, results suggest KNN is the stronger approach as it aligns with the local, spatial nature of property values. Regression remains a useful benchmark, but KNN better captures the role of nearby comparables.

4.3.5 Possible Improvements

One potential limitation of the KNN approach is the risk of spatial data leakage arising from the use of latitude and longitude features. Because nearby properties are often highly similar, a random train–validation split can result in the model effectively “seeing” very similar or adjacent properties during training, leading to overly optimistic performance estimates. In this sense, the model may be learning from properties next door rather than generalizing to new, unseen areas.

A more robust evaluation strategy identified during project presentations would be to implement a spatial holdout, such as splitting the data by community or geographic region, so that entire areas are excluded from training and reserved for validation. This would better test the model’s ability to generalize across different neighbourhoods and provide a more realistic assessment of out-of-sample performance.

5. Random Forest and Gradient Boosting Models

For the Random Forest and Gradient Boosting models, the dataset was partitioned using a time-based split. The training data covered the period from 2021 to 2023, while the validation set corresponded to the year 2024, and the 2025 data was reserved as a final test set. Additionally, the variable *ROLL_YEAR* was incorporated into the analysis to evaluate whether including temporal context could enhance model performance. Although this structure resembles a time series split, the models were not explicitly developed or interpreted as time series models. Rather, this setup was used as a practical approach for validation, and a more rigorous time series framework could be considered as a potential improvement in future work.

Figure 6. Data Split for Random Forest and Gradient Boosting Models.

Figure 7. Frequency of properties per year across Calgary's quadrants.

Exploring the data revealed a consistent increase in the number of properties over time. In some cases, the same property is observed across multiple years, allowing for repeated evaluations;

however, in other instances, certain properties appear only in the validation set, indicating that they were not present in the training data. The only feature explicitly capturing temporal differences in the dataset is *AGE_OF_PROPERTY*, which was computed as the difference between the current year and the property's construction date.

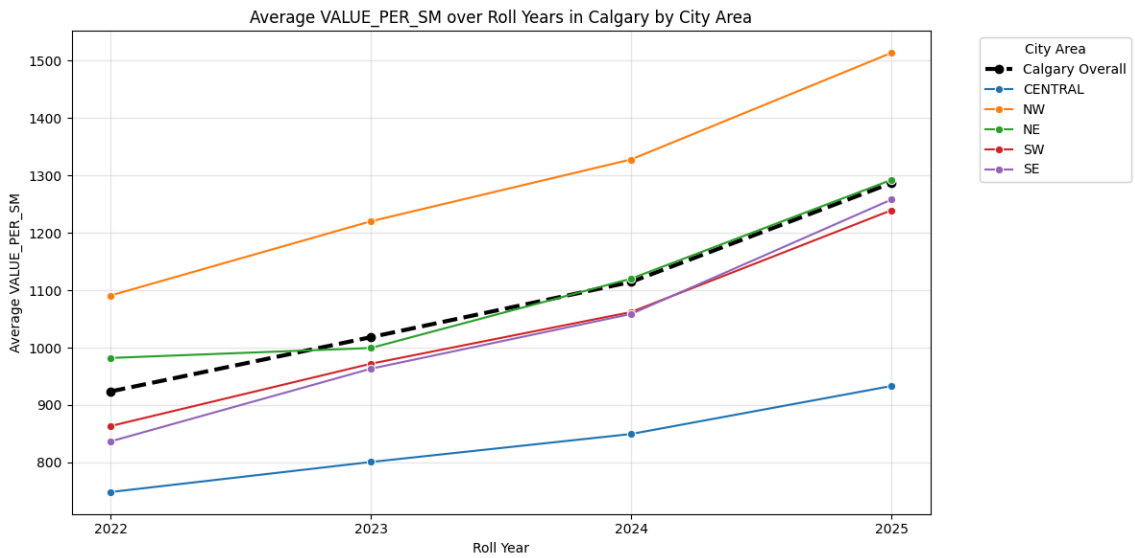

Figure 8. Average Value_per_SM of properties per year across Calgary's quadrants.

In general, when plotting property values per square meter, a clear upward trend is observed over the years. The dashed line represents the city-wide average $/m^2, with all city quadrants following this trend; the most pronounced increase occurs in Calgary NW, while Calgary Central shows the smallest growth. To model these values, both Random Forest and Gradient Boosting algorithms were employed to predict $/m^2 based on a combination of temporal, spatial, and property-specific features.

5.1 Random Forest Model

The Random Forest and Gradient Boosting models were selected not only to predict property values but also to capture the non-linear relationships present in the selected set of features. These models were employed to estimate the $/m^2 of a property using a combination of temporal, spatial, and categorical variables. The feature set included temporal indicators such as *ROLL_YEAR* and *AGE_OF_PROPERTY*; spatial variables like *DIST_FROM_CENTRE_KM*, *center_latitude*, and *center_longitude*; and categorical descriptors including *ASSESSMENT_CLASS*, *PROPERTY_TYPE*, and *CITY_AREA*. Together, these features were chosen to reflect both the physical characteristics of the properties and their geographic context, which are key drivers of property valuation.

Feature Type	Feature Name	Description
Numeric	ROLL_YEAR	Year of property assessment
	AGE_OF_PROPERTY	Age of the property (current year minus construction year)
	DIST_FROM_CENTRE_KM	Distance of the property from the city center in kilometers
	center_latitude	Latitude coordinate of the property's location
	center_longitude	Longitude coordinate of the property's location
Categorical	ASSESSMENT_CLASS	Classification of property for assessment purposes
	PROPERTY_TYPE	Type of property (e.g., residential, commercial)
	CITY_AREA	City quadrant or area where the property is located

The target variable: **VALUE_PER_M2** Continue numerical.

Hyperparameters used for model optimization:

```
n_estimators_list = [5, 50, 250, 400]

rmse_train_list = []
rmse_val_list = []

for n in n_estimators_list:
    rf_pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("rf", RandomForestRegressor(
            n_estimators=n,
            max_depth=25,
            min_samples_split=10,
            min_samples_leaf=3,
            max_features="sqrt",
            random_state=42,
            n_jobs=-1,
            verbose=0
        ))
    ])
```

The initial Random Forest model required a large amount of time to run, in part because the trees were allowed to grow to unlimited depth. To improve efficiency while maintaining predictive performance, the hyperparameters were adjusted to a combination that balanced accuracy and computation time. The final configuration used was:

- n_estimators = 250,
- max_depth = 25,
- min_samples_split = 10,
- min_samples_leaf = 3, and
- max_features = "sqrt".

This setup provided the best predictions without excessively long training times.

5.2 Random Forest Results

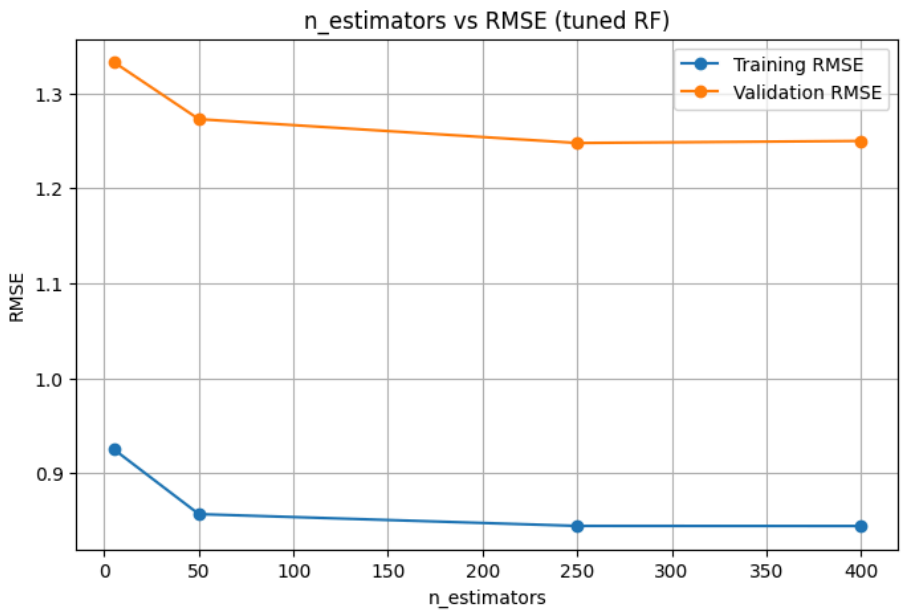

Figure 9. Random Forest - n_estimators vs. RMSE

An `n_estimators` value of 250 was found to yield the lowest RMSE for both the training and validation datasets, indicating an optimal balance between model performance and computational efficiency. Beyond this point, additional trees did not produce significant improvements, suggesting diminishing returns in predictive accuracy.

Train R2:	0.8561
Train RMSE (log):	0.8444
Validation R2:	0.6941
Validation RMSE (log):	1.2475

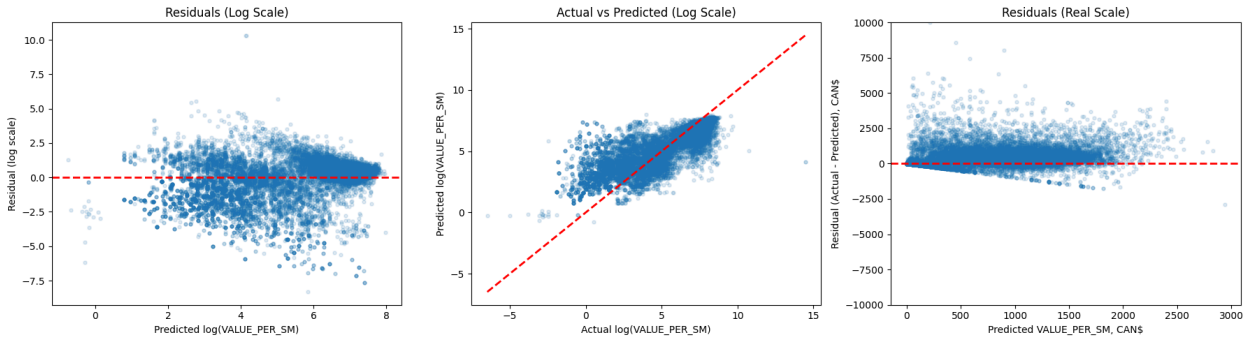

Figure 10. Random Forest a) Residuals Log Scale . b) Actual vs. Predicted (Log Scale) . c) Residuals (Real Scale)

A logarithmic scale of the residuals was required to better capture the wide variation in property values ($/m^2) observed across the city. When analyzing the log-scale residuals, the actual versus predicted values were generally distributed around zero, indicating no strong global bias. However, even in log scale, the residuals ranged approximately from -7.5 to +5 (Figure 10a), with the presence of some outliers that appeared to be legitimate observations rather than data errors. After transforming the residuals back to real values, a noticeable trend remained, suggesting that important explanatory features are still missing from the model (Figure10c).

Model performance metrics further support this observation. The validation log(RMSE) of 1.24 compared to a training log(RMSE) of 0.84 indicates a gap that points to incomplete feature representation. Similarly, the validation R^2 of 0.694 is significantly lower than the training R^2 of 0.856, reinforcing the conclusion that the model is not fully capturing the underlying patterns in the data and could benefit from additional or improved features.

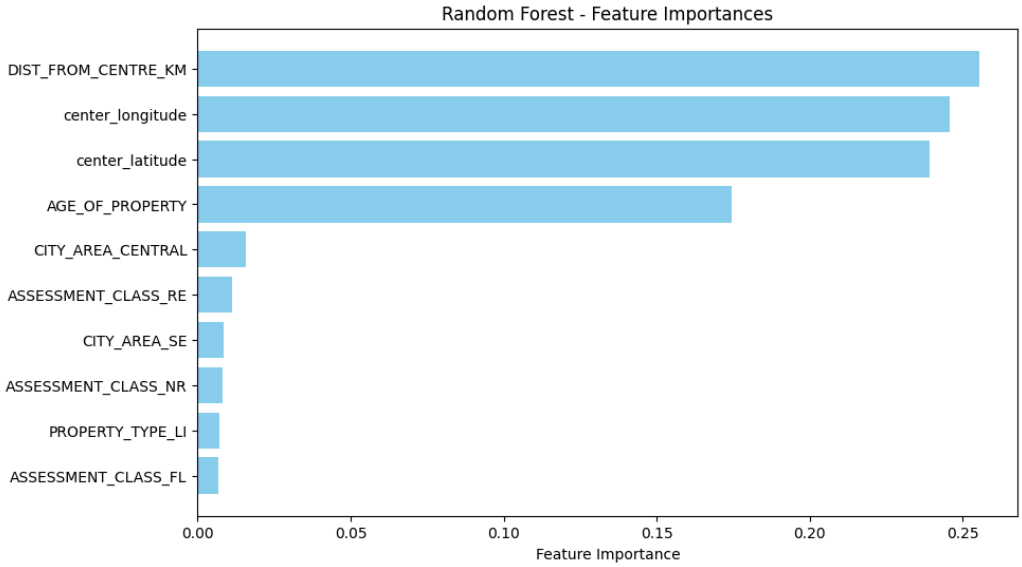

Figure 11. Random Forest Feature Importance

Finally, it was interesting to observe the feature importance results of the model, which showed that geographic variables had the highest influence on predictions. In particular, features such as *DIST_FROM_CENTRE_KM*, *center_longitude*, *center_latitude*, and *AGE_OF_PROPERTY* ranked as the most important contributors. Property-related categorical variables, including *ASSESSMENT_CLASS* and *PROPERTY_TYPE*, also played a role in the model, although with comparatively lower feature importance values in the selected configuration.

5.3 Gradient Boosting Model

Gradient Boosting was explored as an alternative to Random Forest, with the objective of achieving faster convergence while reducing computational time.

```
# ----------------------------
# 1) Apply preprocessing
# ----------------------------
X_train_processed = preprocessor.fit_transform(X_train)
X_val_processed   = preprocessor.transform(X_val)

# ----------------------------
# 2) Gradient Boosting configuration
# ----------------------------
staged_cfg = dict(
    learning_rate=0.1,
    n_estimators=1000,
    max_depth=5
)
gb_staged = GradientBoostingRegressor(random_state=607, **staged_cfg)

# ----------------------------
# 3) Fit the model on processed training data
# ----------------------------
gb_staged.fit(X_train_processed, y_train)
```

Note: Adapted from *Data 607, Lecture 4: Classical Machine Learning*, Dr. Pouria Ramazi.

The same dataset, train–validation split, and feature set used for the Random Forest model were also applied to this model to ensure a consistent and fair comparison.

5.4 Gradient Boosting Results

The model showed continuous improvement in RMSE for both training and validation sets, reaching the best validation RMSE of 1.208 at `n_estimators = 1000`. Testing up to 2000 estimators provided negligible improvement, illustrating diminishing returns. Using early stopping as a guiding principle, the number of estimators was set to 1000, capturing optimal performance without unnecessary computation (Figure XX)

The selected hyperparameters were:

- Learning_rate = 0.1
- n_estimators = 1000
- max_depth=5

Learning rates higher than 0.1 were tested. Although they reduced processing time and initially decreased model error, they increased the risk of missing the global minimum, potentially leading to suboptimal solutions or overfitting. The `max_depth` of the trees was also varied, from very shallow two-layer trees to deeper structures. A `max_depth` of 5 was selected as an optimal point, providing a balance between predictive performance ($R^2 \approx 0.85$ on the validation set) and computational efficiency.

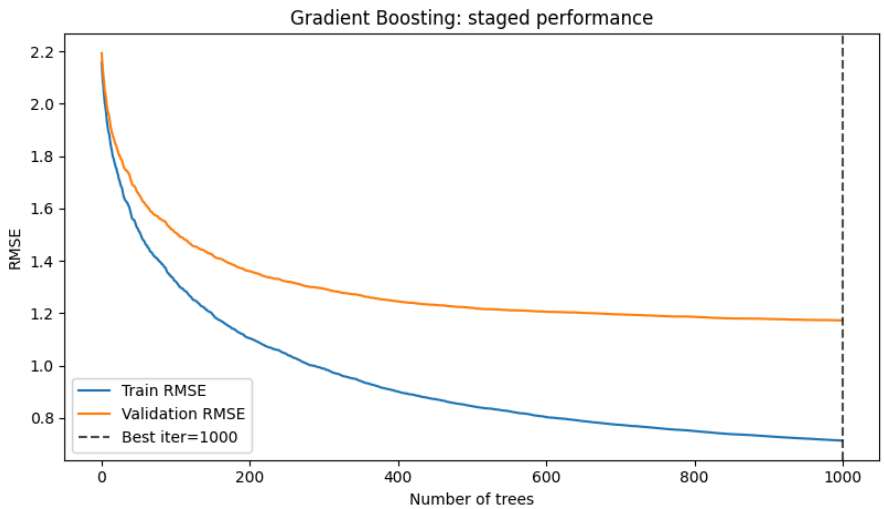

Figure 12. Gradient Boosting - n_estimators (Number of trees) vs. RMSE

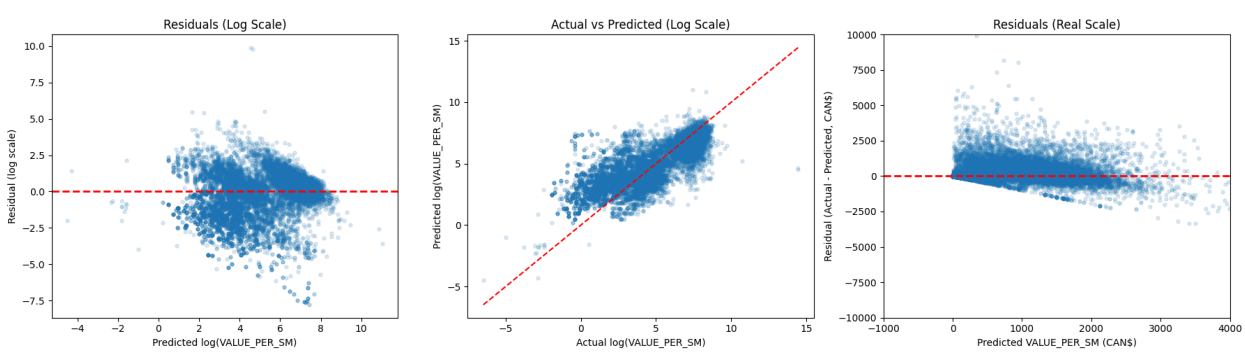

Figure 13. Gradient Boosting. a) Residuals Log Scale . b) Actual vs. Predicted (Log Scale) . c) Residuals (Real Scale)

The residuals on the real scale ranged from approximately -2,500 to 7,500 CAN$. At lower predicted $/m^2 values, the residuals displayed a wider spread, whereas at higher predicted values, the range narrowed to about -2,500 to 2,500 CAN$. This pattern reveals a clear trend in the residuals, indicating that the model is still missing important explanatory features that, if included, could further reduce errors and improve predictive performance.

Similarly to observed in Random Forest, the best fit variables were:

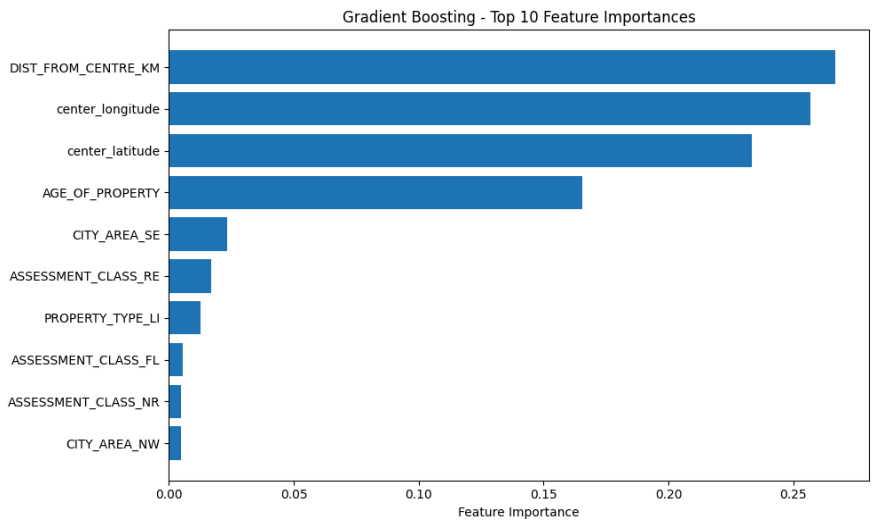

Figure 14. Gradient Boosting Feature Importance

A comparative table between Random Forest and Gradient Boosting model was created:

Model	Train R²	Train RMSE	Validation R²	Validation RMSE
OLS_RidgeCV	0.6012	1.4063	0.5565	1.5023
Random Forest	0.8562	0.8445	0.6941	1.2476
Gradient Boost	0.8974	0.7131	0.7299	1.1723

The current Gradient Boosting model was able to explain a substantial portion of the variance in property values, with a training R^2 of 0.8974 and a training RMSE of 0.7131. On the validation set, the model achieved an R^2 of 0.7299 and an RMSE of 1.1723, outperforming both the OLS RidgeCV and Random Forest models. These results indicate that Gradient Boosting captures the underlying patterns in the data more effectively while maintaining reasonable generalization to unseen properties.

5.5 Limitations and Possible Improvements

1. **Model optimization:** Further hyperparameter tuning could help reduce the current prediction errors.
2. **Features:** Incorporating additional data, such as local economic indicators and census information, may improve predictions, as the current dataset has limited features.
3. **Evaluation & modeling:** Applying time series techniques—such as lag features and rolling averages—could better capture year-over-year variations in property prices.

6. Conclusions

SINGLE YEAR EVALUATION:

1. A single feature (nearby market value) drives most predictive power. Removing it causes a major drop in performance → most of the signal is local.
2. KNN outperforms regression in predictive accuracy. Using actual nearby observations beats trying to approximate the relationship linearly.
3. Regression is useful for interpretation, not performance. It clearly shows what matters, but not necessarily the best way to predict.
4. Property-specific features have limited incremental value. Once local market context is included, features like property type or age add relatively little.
5. The problem is fundamentally spatial, not structural. Value is better understood as a function of where a property is rather than what it is.

MULTIPLE YEARS EVALUATION:

1. **Splitting the data by year** for train/validation **did not reduce the observed error**, whether done within the same year or ignoring the year split
2. Adding ROLL_YEAR as a feature did not improve model performance
3. Most significant features across models:
    - **Time**: Property age
    - **Location**: Latitude, longitude, distance from city center

    Used in spatiotemporal and hedonic models for property price prediction [1],[2]

4. In Gradient Boosting, the combination of n_estimators, max_depth, and learning_rate led to significant improvements over the Random Forest model. Gradient Boosting converged more quickly, but it must be used cautiously, as overly aggressive settings of any of these parameters can cause overfitting. Techniques such as **early stopping** were employed to prevent this and ensure stable, generalizable performance.
5. Gradient Boosting and its variations (e.g., XGBoost) could improve R^2 and RMSE, potentially by incorporating local economic indicators such as Bank of Canada interest rates, GDP, etc.

7. References

1. **Dubé, J., & Legros**, D. (2014). Spatial econometrics and the hedonic pricing model: What about the temporal dimension? *Journal of Property Research, 31*(4), 333–350. https://doi.org/10.1080/09599916.2014.913655
2. **Muto, S., Morishige,** C., & Matsumoto, T. (2023). Hedonic real estate price estimation with a spatiotemporal geostatistical model. *Asia-Pacific Journal of Regional Science*. https://doi.org/10.1007/s43071-023-00039-w
3. **Ramazi, P. (2025).** *Classical Machine Learning* [Lecture 4, Data 607].